



INSTITUTE OF ENGINEERING AND MANAGEMENT, KOLKATA
ELECTRONICS AND COMMUNICATION ENGINEERING DEPARTMENT

PROJECT MODULE - 6

IVI IN-VEHICLE INFOTAINMENT SYSTEMS

AUTOMOTIVE MEDIA PLAYER

ANDROID AUTOMOTIVE IMPLEMENTATION REPORT

Android Automotive media-player UI, media-command propagation, simulated Media Service—Audio HAL flow, emulator testing, physical-device deployment attempt and Logcat system behavior

NAME	ENROLLMENT NUMBER	SEC & ROLL NO.
SAGNIK BHATTACHARYYA	12023052003048	A - 47

. CONTENTS

S.NO	TOPICS	PAGE NO.
1	Executive Summary	3
2	Problem Statement, Objectives and Engineering Requirements	4
3	Overall Android Studio / Automotive Architecture	5
4	Conceptual IVI Media Stack and Command Model	6
5	Android Automotive Emulator and Development Configuration	7
6	Media Player UI and User Interaction	8
7	Java Activity and Event Handling	9
8	Media Service and Audio Framework Simulation	10
9	Logcat Configuration and Message-Flow Troubleshooting	11
10	Playback State and Seek Control	12
11	Integration Testing and Expected Status Behavior	13–14
12	End-to-End Implementation Representation	15
13	Test Cases and Observed Results	16
14	Development Process and Design Decisions	17
15	Validation, Limitations and Engineering Assumptions	18
16	Future Scope and Advanced Development Roadmap	19
17	Conclusion	20

REPORT ORGANIZATION

The report follows the structure of the reference project-report format: executive summary, engineering requirements, architecture, implementation, testing, validation, future scope and conclusion. The revised edition removes all running headers and footers and uses the available evidence directly inside the technical sections.

Evidence category	Included in revised report
Application UI	Final Automotive Media Player screen
Development	Android Studio Java implementation screenshots
Emulator	Running Devices and Automotive runtime evidence
Troubleshooting	Emulator storage error and physical-device installation restriction
Architecture	Conceptual IVI command-flow diagrams and tables
Testing	Command/state test matrix and observed behavior

1. Executive Summary

The Automotive Media Player is a simplified Android Automotive In-Vehicle Infotainment (IVI) demonstration developed in Android Studio using Java and XML. The implementation concentrates on the media-control portion of an IVI system and makes the relationship between user input, application logic, conceptual service layers and runtime observation visible.

The application provides an automotive-oriented interface containing track information, artist information, a playback state, a seek/progress control and controls for PLAY, PAUSE, STOP, NEXT and PREVIOUS. The visible state changes make the interaction immediately observable during a demonstration.

The software-layer model represents the conceptual sequence **User Input** → **Media Player UI** → **Media Service** → **Audio Framework** → **Audio HAL**. The corresponding messages are written by the application to Logcat using the MEDIA_PLAYER tag. These messages are intentionally a simulation of the lower-layer propagation; they do not constitute a production Android Audio HAL implementation.

The project was built and tested in an Android Automotive emulator. The final Automotive screen was successfully demonstrated, while an additional physical Xiaomi-device installation attempt was recorded separately. The phone deployment evidence shows that installation was blocked by a device security restriction, so the Automotive emulator remains the primary integration environment documented by this report.

The result is an educational prototype that demonstrates Android application development, automotive UI design, event handling, diagnostic logging, layered IVI reasoning and repeatable emulator-based testing.

Project outcome	Status
Automotive media-player UI	Implemented and demonstrated
Playback controls	Implemented
Playback state model	Implemented
Conceptual Media Service / Audio Framework / Audio HAL flow	Represented through application logs
Automotive emulator execution	Demonstrated
Physical Xiaomi deployment	Attempted; installation restricted by device security
Production audio / vehicle hardware integration	Outside current scope

2. Problem Statement, Objectives and Engineering Requirements

The Module 6 project brief calls for a basic Android Automotive application, deployment and testing through an Android Emulator or Raspberry Pi environment, and logging of system behavior. The implemented option is a music-player prototype with a conceptual Media Service–Audio HAL command path.

Engineering problem. A driver-facing media control is a high-level software event. In a complete IVI stack, that event is handled by several software layers before it can affect audio output. For an academic prototype, the important requirement is to make this propagation understandable and observable without requiring vehicle hardware.

Implemented objectives. The project creates the media UI, supports the primary playback commands, exposes playback state, provides seek control, emits MEDIA_PLAYER diagnostic messages, represents the conceptual lower-layer flow and validates the workflow in an Automotive emulator.

Scope boundary. Media Service, Audio Framework and Audio HAL are represented as conceptual layers in this prototype. The application-generated messages are evidence of the demonstration logic, not proof of platform-level Android Automotive service or HAL implementation.

Requirement	Implemented behaviour	Evidence
User interaction	Buttons and seek bar accept input	Final Automotive UI
State visibility	Ready / Playing / Paused / Stopped	Application status field
System logging	MEDIA_PLAYER diagnostic messages	Logcat workflow
Layered command model	UI → service → framework → HAL representation	Architecture + logs
Automotive test	Application executed in Automotive emulator	Running-device evidence
Physical-device attempt	Wireless deployment attempted on Xiaomi	Installation restriction screenshot

ENGINEERING PRINCIPLE

The prototype deliberately separates what is implemented from what is simulated. This keeps the report technically defensible: the application implements the interface, state model and logs, while the lower audio-stack stages are explanatory representations.

3. Overall Android Studio / Automotive Architecture

The project is organized as a conventional Android application and executed inside an Android Automotive virtual device. Android Studio provides the development environment, Gradle builds the application, Java contains the event-handling logic and XML defines the interface. The Automotive emulator provides the runtime environment in which the application is rendered.

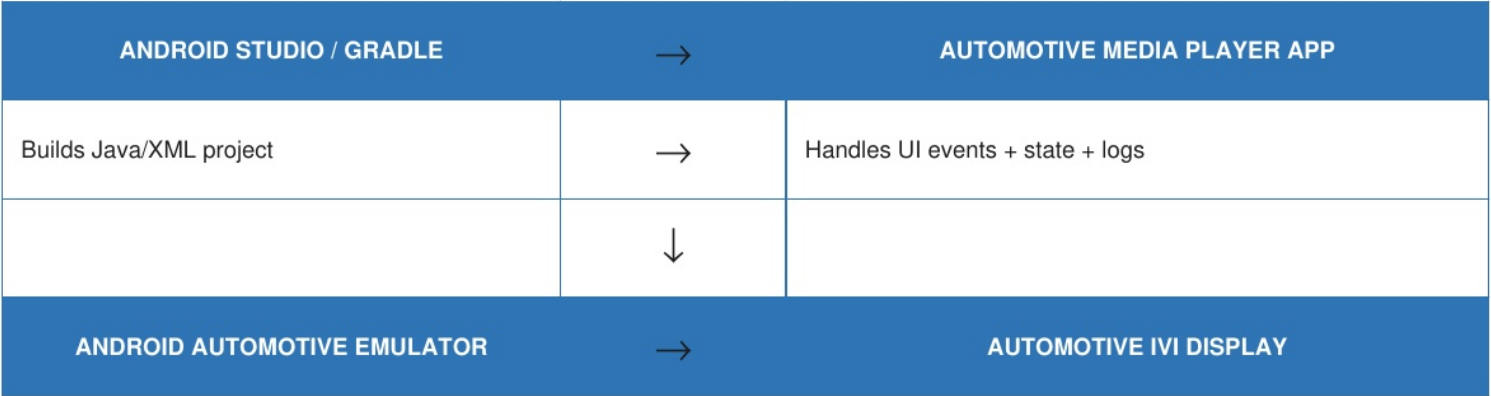


Figure 3.1. Development-to-runtime organization of the Automotive Media Player.

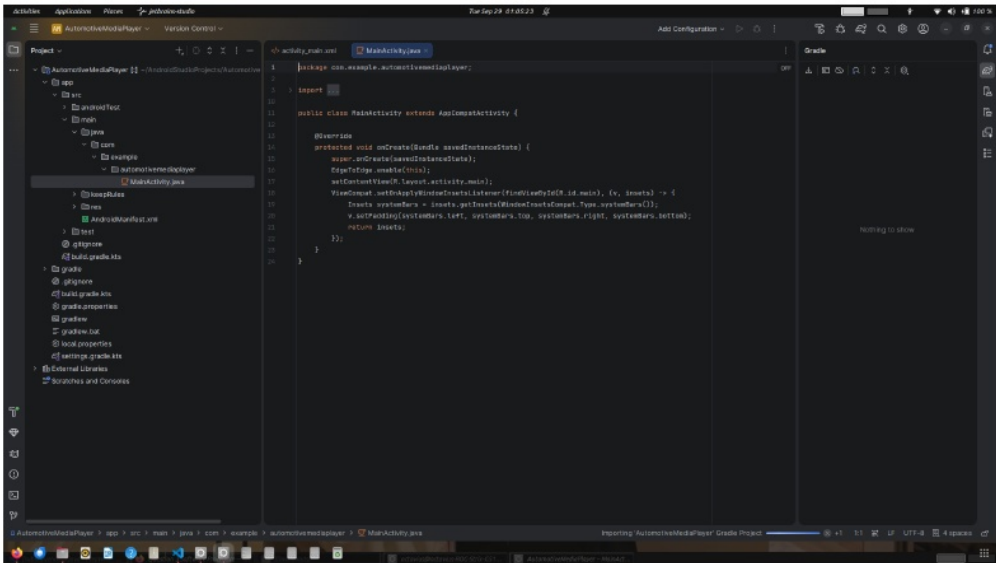


Figure 3.2. Android Studio project environment used during implementation.

Subsystem	Input	Output	Purpose
UI layer	Touch/click	Command event	Collect media interaction
Application logic	Command event	State + logs	Update state and trace action
Service model	Application command	Service message	Represent Media Service handling
Audio-stack model	Service message	Framework/HAL logs	Represent lower-layer propagation
Emulator	Built APK	Running application	Repeatable automotive test environment

4. Conceptual IVI Media Stack and Command Model

The central model is that a driver-facing media command is not identical to a hardware operation. The interface creates an event; application logic interprets it; conceptual service and audio layers represent the path toward playback.

Layer	Responsibility	Example for PLAY
User	Provides intent	Driver presses PLAY
Media Player UI	Captures interaction	onClick event generated
Media Service	Represents media request handling	PLAY request received
Audio Framework	Represents audio command propagation	PLAY command sent
Audio HAL	Represents hardware-facing stage	Playback started message
Observation	Makes sequence visible	MEDIA_PLAYER Logcat entries



Figure 4.1. Conceptual command propagation used by the prototype.

For PLAY the application records: **User input: PLAY** → **Media Service: PLAY request received** → **Audio Framework: PLAY command sent** → **Audio HAL: Playback started**. PAUSE and STOP use the same explanatory pattern with the corresponding state and log messages.

TRACEABILITY

The value of the model is traceability. A single button press can be followed from the visible interface to a sequence of application-generated diagnostic messages.

5. Android Automotive Emulator and Development Configuration

The Automotive emulator was used as the primary runtime because it provides a vehicle-oriented Android environment without requiring access to a physical vehicle. The selected virtual device was an Automotive Distant Display with Google Play, using an Android 13 / API 33 x86_64 system image.

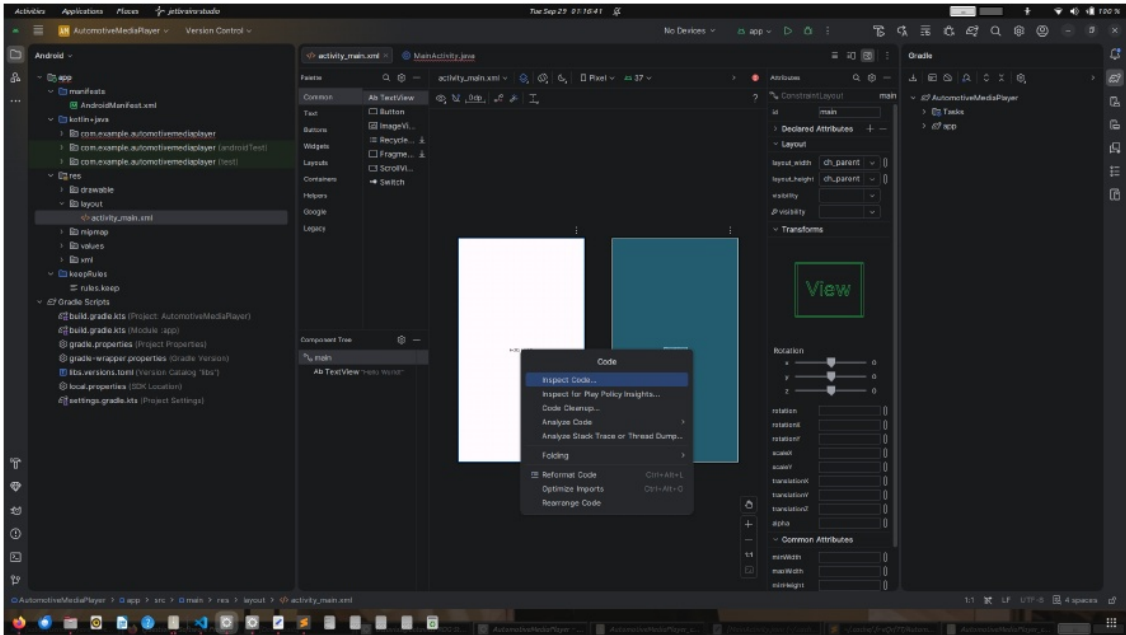


Figure 5.1. Android Studio emulator/device configuration stage.

Configuration item	Project setting
IDE	Android Studio Quail 4 2026.1.4 Patch 1
Language	Java
UI	XML layout
Virtual device	Automotive Distant Display with Google Play
Android API	33 / Android 13 (Tiramisu)
Architecture	x86_64 system image
Primary validation	Automotive emulator

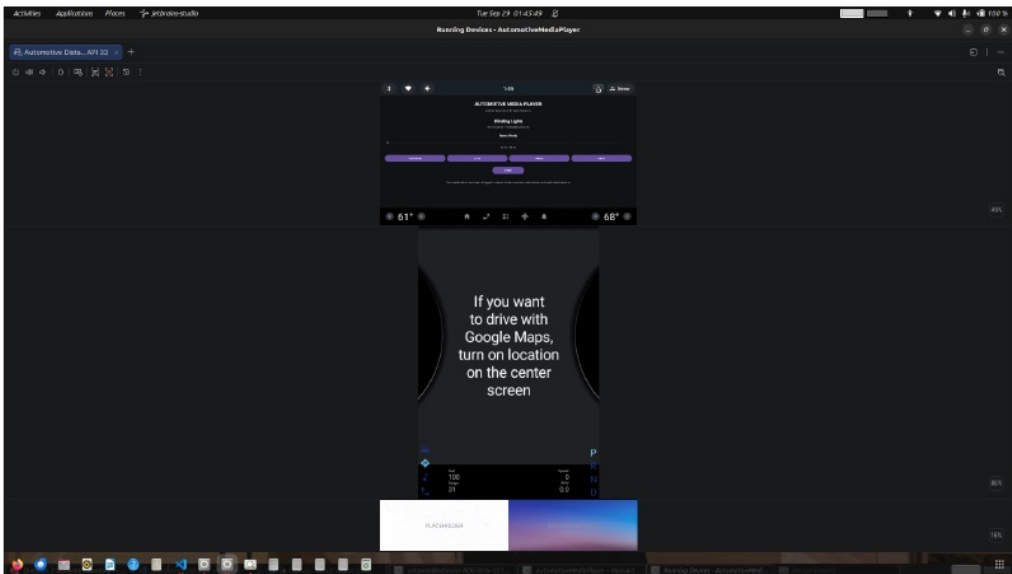


Figure 5.2. Android Studio Running Devices view showing the Automotive runtime.

The emulator provided the repeatable environment used to demonstrate the final UI and interaction model. It also made it possible to distinguish application behavior from host-environment problems encountered during setup.

6. Media Player UI and User Interaction

The interface was designed as a compact automotive media screen. It presents the application title, track information, artist information, current state, seek control and the main playback commands. The objective is not visual complexity; it is clear interaction and direct observability of the state model.

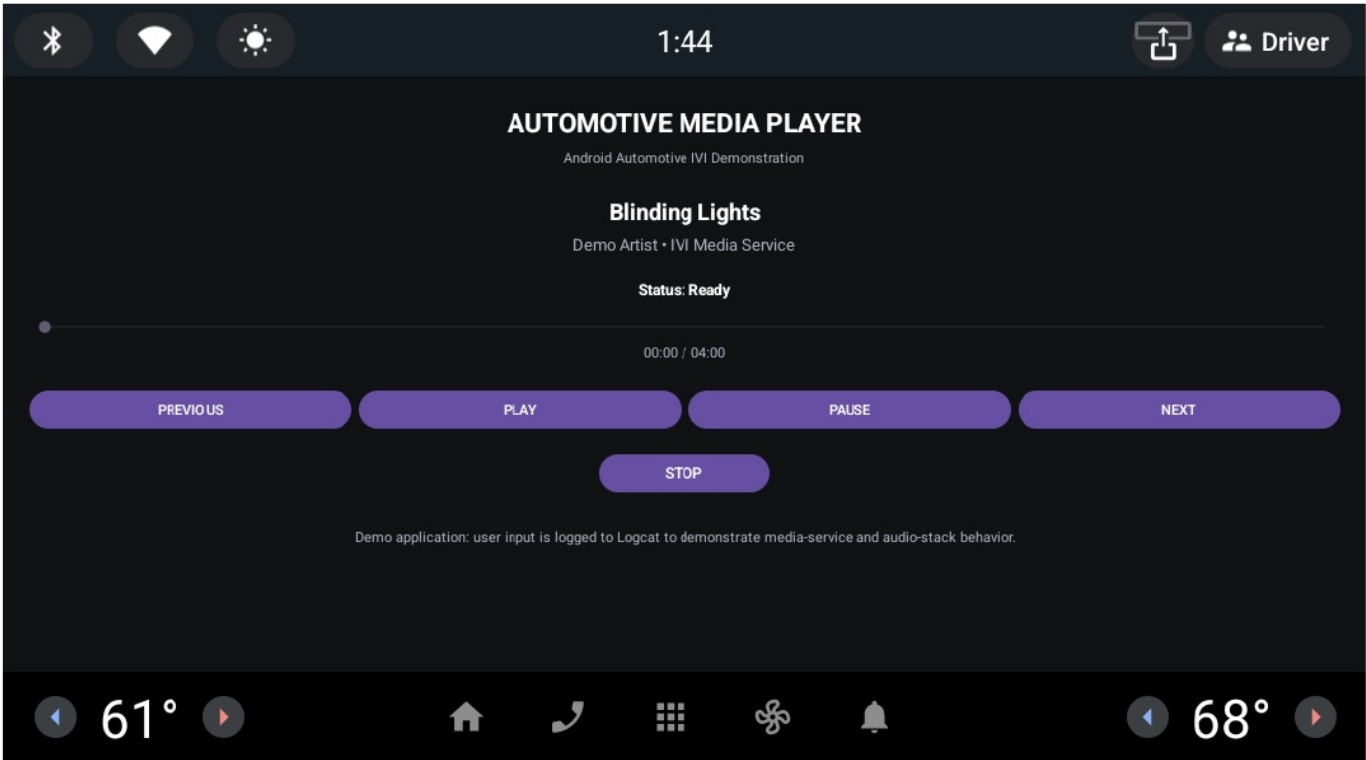


Figure 6.1. Final Automotive Media Player interface running in the Automotive environment.

UI element	Function	Expected effect
Track title	Displays current media item	Identifies active track
Artist / service text	Provides media metadata and layer context	Clarifies demonstration scope
Status	Displays Ready / Playing / Paused / Stopped	Makes state transition visible
Seek bar	Represents playback position	Updates progress value
PLAY	Starts simulated playback state	Status → Playing
PAUSE	Pauses simulated playback state	Status → Paused
STOP	Stops simulated playback state	Status → Stopped
NEXT / PREVIOUS	Records navigation of media command	Event written to Logcat

The screen also retains the automotive display environment around the application, which helps demonstrate that the project is intended for an IVI-style runtime rather than a conventional phone-only UI.

7. Java Activity and Event Handling

The Java activity is responsible for connecting the XML controls to event handlers and maintaining the demonstration state. The implementation uses straightforward Android view references and application logging so that the behavior remains easy to inspect.

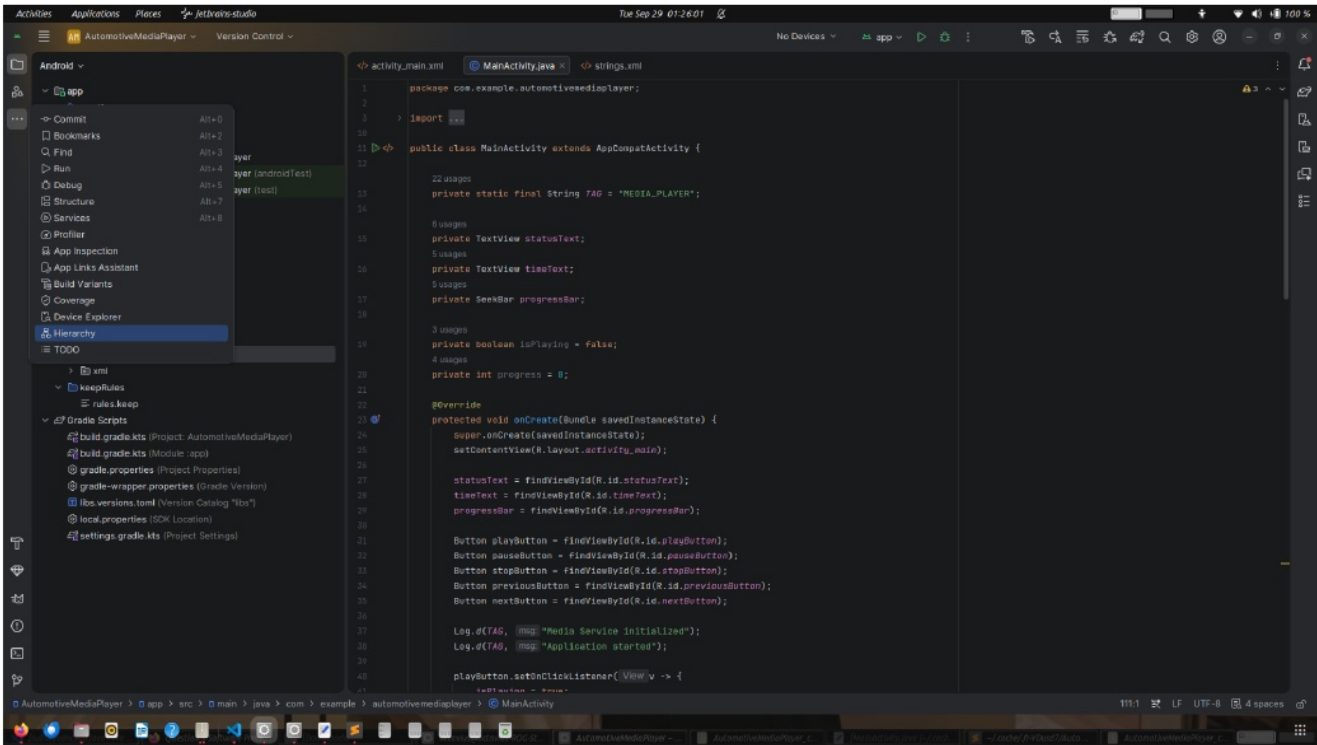


Figure 7.1. MainActivity.java showing state variables, UI references and MEDIA_PLAYER logging.

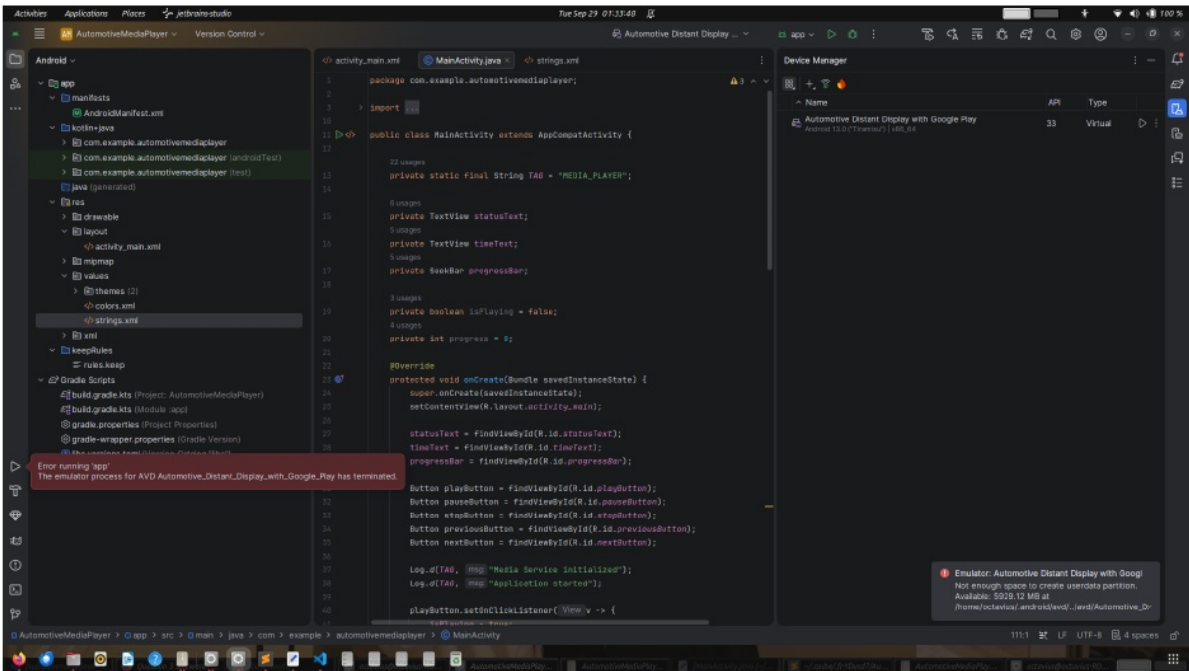
The activity initializes the interface in **onCreate()**, obtains references to the status text, seek bar and control buttons, and attaches listeners. Each listener performs two related actions: it updates the visible state or progress and writes a diagnostic message.

Event	Application action	Diagnostic intent
PLAY	Set state to Playing	Trace request and playback start
PAUSE	Set state to Paused	Trace pause propagation
STOP	Set state to Stopped	Trace stop propagation
NEXT	Record next-track event	Trace command reception
PREVIOUS	Record previous-track event	Trace command reception
SEEK	Update progress/time representation	Trace seek event

The code is intentionally compact. A production implementation would normally delegate playback to a media engine or platform media service instead of simulating the lower layers directly.

8. Media Service and Audio Framework Simulation

The application models the lower audio stack through log messages rather than by implementing platform-level services. This distinction is important because the project requirement is to demonstrate message flow and system behavior, not to reproduce the complete Android Automotive audio architecture.



A representative PLAY sequence is shown below.

Stage	Representative message	Meaning
User	User input: PLAY	The user initiated the command
Media Service	Media Service: PLAY request received	Application represents service-level handling
Audio Framework	Audio Framework: PLAY command sent	Application represents framework propagation
Audio HAL	Audio HAL: Playback started	Application represents the hardware-facing stage

TECHNICAL LIMITATION

The Audio HAL line is generated by the application. It is not a call into a real vendor Audio HAL, and no physical speaker output is claimed. The same applies to the conceptual Media Service and Audio Framework lines.

This approach is useful for an academic demonstration because it gives each layer an explicit and testable representation while keeping the implementation small enough to understand and modify.

9. Logcat Configuration and Message-Flow Troubleshooting

Logcat is the primary runtime-observation mechanism for the project. The dedicated tag **MEDIA_PLAYER** makes it easier to distinguish the application's messages from the much larger Android system log.

Step	Action	Expected observation
1	Select the running Automotive emulator	Correct device appears in Android Studio
2	Open Logcat	Runtime messages are visible
3	Filter for MEDIA_PLAYER	Project-specific messages are isolated
4	Press PLAY / PAUSE / STOP	Corresponding command sequence appears
5	Use NEXT / PREVIOUS / SEEK	Additional events are logged
6	Compare UI state and logs	Visible behavior matches trace

Physical-device runtime evidence. The following photographs show the application running on the physical phone in multiple interaction states. They provide direct visual evidence of the same media-player interface outside the Automotive emulator.

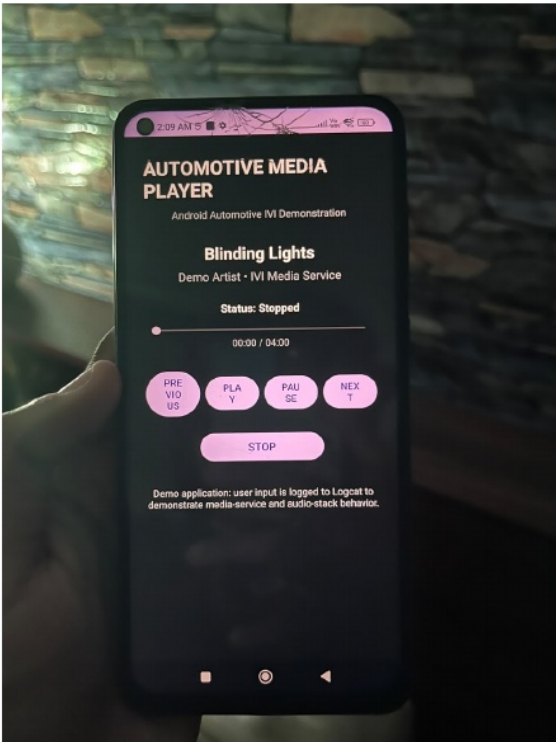


Figure 9.1. Phone — Stopped state.

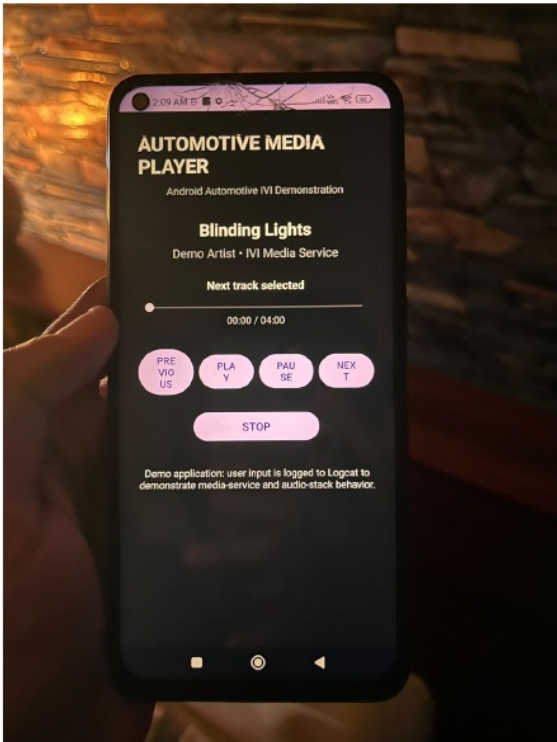


Figure 9.2. Phone — Next track selected.

These phone photographs demonstrate that the interface responds to user commands and exposes the resulting state or track-selection message directly on the device.

TROUBLESHOOTING PRINCIPLE
The report separates successful application execution from environment-specific deployment issues. The phone photographs document runtime behavior; the separate installation restriction screenshot remains deployment evidence.

10. Playback State and Seek Control

The application uses a compact state model so that media interaction can be observed without implementing a complete audio engine. The main states are Ready, Playing, Paused and Stopped.

READY	PLAY	PLAYING
PLAYING	PAUSE	PAUSED
PAUSED	PLAY	PLAYING
ANY ACTIVE STATE	STOP	STOPPED

Figure 10.1. Simplified playback-state transition model.

PLAY. The state becomes Playing and the conceptual service/framework/HAL sequence is logged. **PAUSE.** The state becomes Paused and a corresponding pause trace is generated. **STOP.** The state returns to Stopped.

Seek control. The SeekBar represents playback position. Moving the control updates the progress value and records the event, giving the prototype a continuous-looking media control while retaining a simple event-driven implementation.

Control	State / data effect	Logging effect
PLAY	Playing	PLAY propagation sequence
PAUSE	Paused	PAUSE propagation sequence
STOP	Stopped	STOP propagation sequence
SEEK	Progress value updated	SEEK event recorded

A full media engine would add decoding, buffering, duration management, audio focus and media-session behavior. Those features are reserved for the future-development section.

11. Integration Testing and Expected Status Behavior

Integration testing checks the complete path from the Android UI to the visible status and the corresponding conceptual Logcat trace. The normal condition is the application at launch; active conditions are generated by user commands.

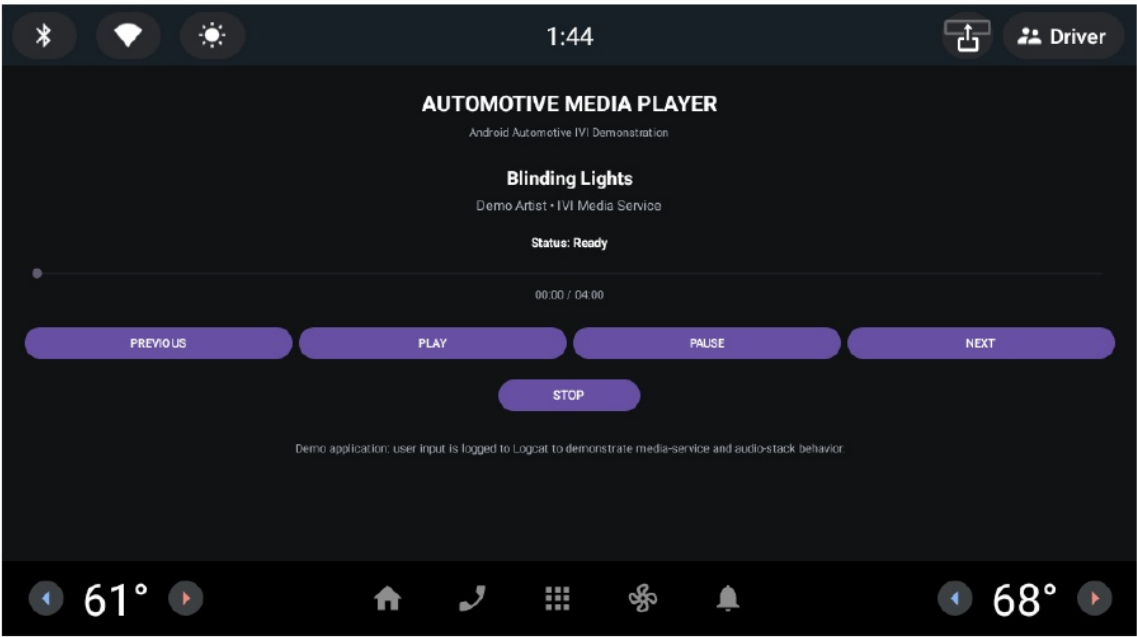


Figure 11.1. Final running Automotive Media Player used as the baseline integration state.

Command	UI result	Conceptual lower-layer result
PLAY	Status = Playing	PLAY request → framework → HAL start
PAUSE	Status = Paused	PAUSE request → framework → HAL pause
STOP	Status = Stopped	STOP request → framework → HAL stop
NEXT	Next-track event recorded	NEXT request logged
PREVIOUS	Previous-track event recorded	PREVIOUS request logged
SEEK	Progress/time updated	SEEK request logged

The test objective is consistency rather than audio fidelity. For every command, the visible state or progress representation should agree with the diagnostic message sequence generated by the application.

PASS CRITERION

A command is considered functionally demonstrated when the UI accepts the input, the expected visible state/progress changes, and the matching MEDIA_PLAYER diagnostic sequence is produced.

11. Integration Testing and Expected Status Behavior — Physical Device Evidence

The physical-phone test provides direct visual evidence of the application operating on the device. Five supplied photographs document different interaction states: Stopped, Next track selected, Paused, Playing and Previous track selected.

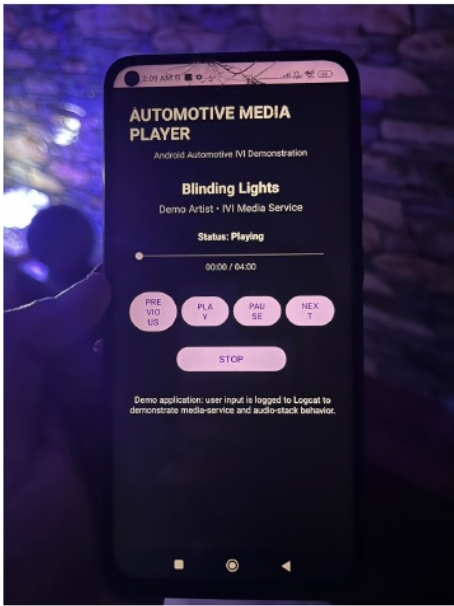


Figure 11.2. Phone — Playing.

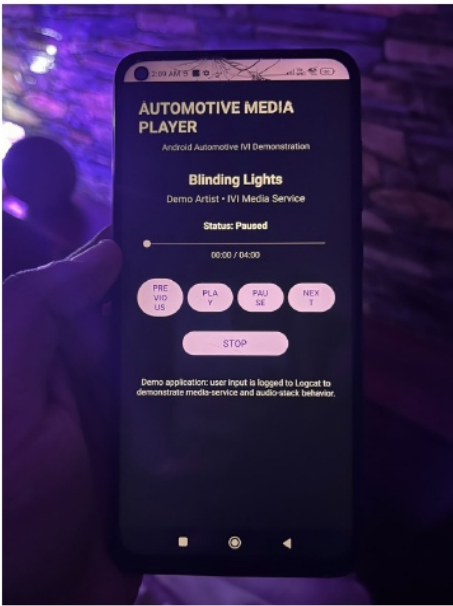


Figure 11.3. Phone — Paused.



Figure 11.4. Phone — Previous track selected.

The screenshots show the same media-player interface in distinct states and therefore strengthen the runtime validation of the control logic. They are photographs of the physical phone rather than emulator screenshots.

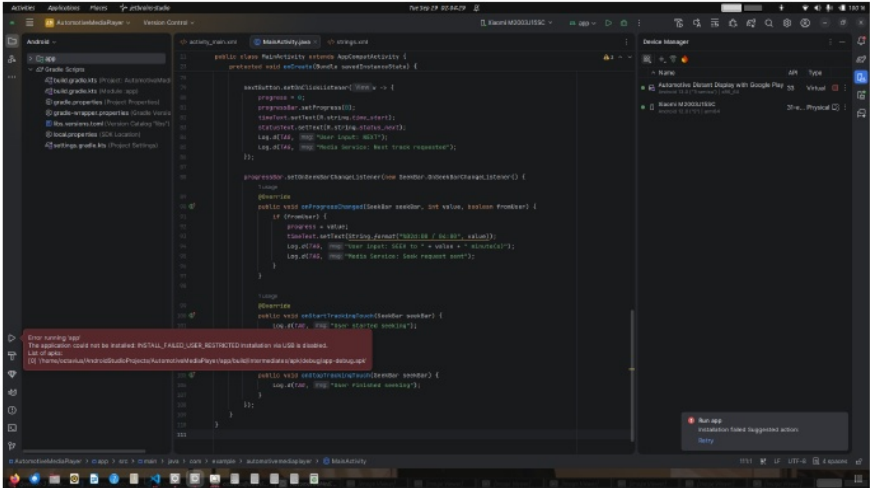


Figure 11.5. Physical Xiaomi-device installation restriction recorded during an earlier deployment attempt.

The installation-restriction screenshot is retained as a separate deployment/troubleshooting record. It does not replace the successful phone-runtime photographs above.

Evidence	Observed result	Interpretation
Phone — Playing	Status: Playing	PLAY interaction demonstrated
Phone — Paused	Status: Paused	PAUSE interaction demonstrated
Phone — Stopped	Status: Stopped	STOP interaction demonstrated
Phone — Next / Previous	Track-selection messages	Navigation controls demonstrated
Automotive emulator	Application displayed	Automotive integration environment demonstrated
Installation restriction screenshot	Installation blocked at one attempt	Separate deployment issue

12. End-to-End Implementation Representation

The project can be represented as a complete educational pipeline from the driver-facing interaction to observable software behavior. The following sequence connects the architecture, Java implementation and runtime evidence.

1. USER
Presses PLAY / PAUSE / STOP / NEXT / PREVIOUS
2. MEDIA PLAYER UI
Android View event listener receives the interaction
3. APPLICATION LOGIC
Updates state, seek position and diagnostic output
4. MEDIA SERVICE — CONCEPTUAL
Represents receipt of the media request
5. AUDIO FRAMEWORK — CONCEPTUAL
Represents command propagation toward audio output
6. AUDIO HAL — CONCEPTUAL
Represents hardware-facing stage through application log
7. OBSERVATION
Visible state + MEDIA_PLAYER Logcat trace

Figure 12.1. End-to-end representation of the implemented demonstration.

The value of this representation is that it gives a common vocabulary for discussing the prototype. The user sees a media-control interface; the developer sees an event-driven Android activity; the IVI architecture discussion sees a layered media path; and the tester sees a sequence that can be verified through state and logs.

Evidence	What it proves
Final Automotive screen	Application rendered in the target-style runtime
Java source screenshot	Event-handling and logging implementation
Running Devices screenshot	Automotive device execution environment
Storage error screenshot	Troubleshooting record
Physical Xiaomi screenshot	Separate deployment restriction record

13. Test Cases and Observed Results

The test matrix focuses on deterministic application behavior. The objective is to confirm that each control generates the intended visible state or event and that the corresponding diagnostic path is available.

TC ID	Action	Expected result	Observed / documented result
TC-01	Launch app	Status = Ready	Ready state shown
TC-02	Press PLAY	Status = Playing + PLAY trace	Implemented in event handler
TC-03	Press PAUSE	Status = Paused + PAUSE trace	Implemented in event handler
TC-04	Press STOP	Status = Stopped + STOP trace	Implemented in event handler
TC-05	Press NEXT	Next event logged	Implemented
TC-06	Press PREVIOUS	Previous event logged	Implemented
TC-07	Move SeekBar	Progress value changes	Implemented
TC-08	Run Automotive emulator	Application displayed	Demonstrated
TC-09	Install on Xiaomi	Application installs	Blocked by device restriction

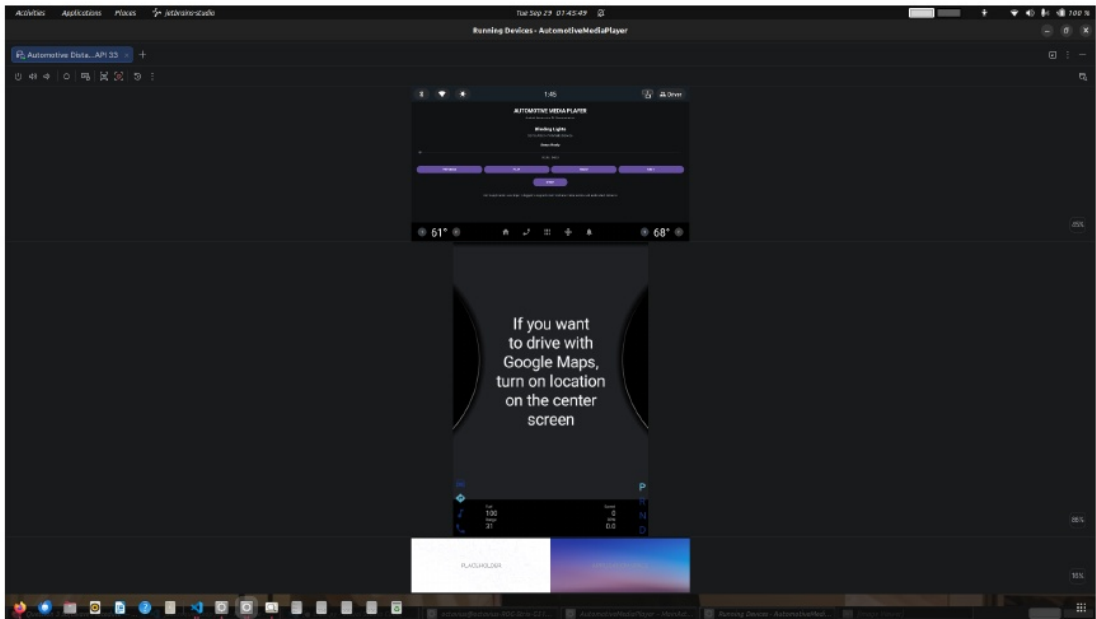


Figure 13.1. Running Devices evidence associated with the Automotive integration test.

The test results show that the principal educational workflow is reproducible in the Automotive emulator. The physical-device test did not reach application execution because installation was stopped before the app could be launched.

The implementation was developed incrementally so that each layer could be verified before adding the next. This reduced ambiguity during troubleshooting and made the final report traceable to concrete evidence.

Stage	Work completed	Reason
1	Create Android project	Establish standard Gradle/Java/XML structure
2	Design automotive-oriented UI	Provide visible media interaction
3	Connect Java listeners	Make controls functional
4	Add state model	Make commands observable
5	Add MEDIA_PLAYER logs	Make command flow traceable
6	Configure Automotive AVD	Create repeatable runtime
7	Resolve emulator storage issue	Enable successful runtime execution
8	Run and inspect application	Validate final UI
9	Attempt physical-device deployment	Check an additional execution path
10	Document limitations	Keep implementation claims technically precise

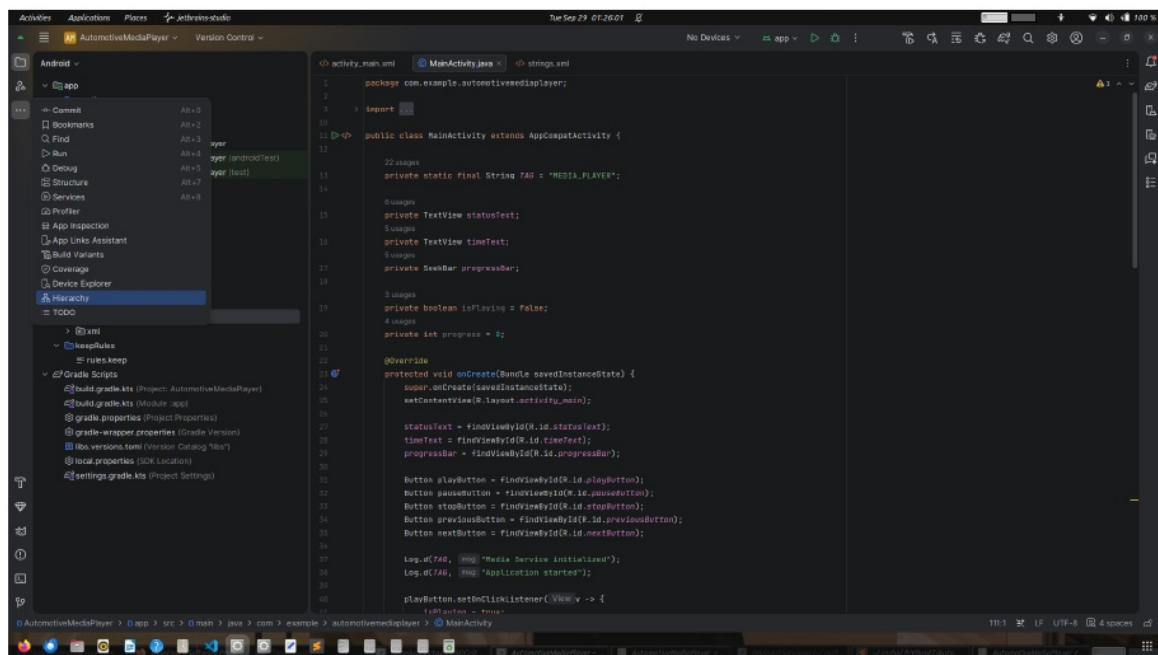


Figure 14.1. Development evidence showing the Java implementation and logging logic.

Design decision: simulation over production complexity. A full Android Automotive audio implementation would require substantially more platform integration. The project instead makes the command path visible through application-level logging. This keeps the demonstration focused on the Module 6 learning objective.

Design decision: emulator-first validation. The Automotive emulator provides a stable and repeatable environment. Physical-device deployment is treated as an additional test path rather than the primary evidence source.

15. Validation, Limitations and Engineering Assumptions

Validation is based on build success, application rendering, interaction design, state visibility and the documented emulator execution. The report deliberately avoids treating simulated messages as hardware-level proof.

Area	Validated	Limitation / assumption
UI rendering	Automotive media screen displayed	No vehicle HMI certification claimed
Controls	Buttons and seek interaction implemented	No physical steering-wheel control
Playback state	Ready / Playing / Paused / Stopped represented	No real audio stream required
Logging	MEDIA_PLAYER diagnostic path designed	Messages are application-generated
Audio HAL	Conceptual flow represented	No production/vendor HAL implementation
Vehicle integration	Not implemented	No CAN bus or physical speakers
Phone deployment	Attempted	Installation blocked by device security

Validation conclusion. The prototype satisfies the educational implementation scope demonstrated in the supplied evidence: Android application development, Automotive UI, interaction handling, conceptual media-stack propagation, runtime-oriented logging and emulator-based execution.

IMPORTANT ASSUMPTION

The project is a software demonstration. It should not be interpreted as a production-ready automotive infotainment system, a validated vehicle audio controller, or a real Audio HAL implementation.

Deployment evidence note. The Xiaomi screenshot documents an installation restriction rather than a failed application runtime. The distinction is important because the device did not reach the application execution stage.

16. Future Scope and Advanced Development Roadmap

The current prototype is intentionally small, but its architecture can be extended in several directions without discarding the existing UI and command model.

Future stage	Extension	Expected engineering value
1	Real audio playback	Move from state simulation to actual media output
2	Media3 / MediaSession integration	Use standardized Android media APIs
3	Audio focus	Coordinate music with navigation and other audio sources
4	Multi-track playlist	Give NEXT/PREVIOUS real track semantics
5	Navigation mockup	Demonstrate concurrent IVI functions
6	Incoming-call interruption	Pause/resume media around telephony events
7	Voice commands	Map speech commands to the same event interface
8	Vehicle-specific audio routing	Explore platform-level automotive integration
9	Physical hardware	Validate on automotive development hardware

Advanced interruption model. A future implementation could demonstrate: Music Playing → Navigation Prompt / Incoming Call → Audio Focus Request → Music Paused or Ducked → Interruption Ends → Audio Focus Released → Music Resumed.

Advanced development roadmap. Prototype UI → real playback → MediaSession/Media3 → audio focus → multi-app IVI integration → vehicle-specific audio services → hardware validation.

EXTENSIBILITY

The current separation between UI events, application state and conceptual lower-layer logging provides a clear place to replace simulated stages with real Android Automotive services in a future version.

17. Conclusion

The Automotive Media Player project provides a practical introduction to Android Automotive and In-Vehicle Infotainment software development. It converts a simple media-control requirement into a complete educational demonstration containing a user interface, Java event handling, a conceptual service/audio-stack model, runtime-oriented logging and emulator-based integration testing.

The core implementation demonstrates the sequence **USER INPUT** → **MEDIA PLAYER UI** → **MEDIA SERVICE** → **AUDIO FRAMEWORK** → **AUDIO HAL** → **OBSERVABLE STATE / LOGCAT**. The most important engineering result is traceability: a user interaction can be connected to a visible state change and to a diagnostic command sequence.

The project also establishes a clear boundary between prototype behavior and production implementation. The Media Service, Audio Framework and Audio HAL messages are simulated through application logging; they are not evidence of real platform-level hardware integration. Likewise, the Automotive emulator validates the software workflow but cannot replace vehicle-level testing.

Physical-device deployment was additionally attempted on a Xiaomi device. The supplied evidence shows that installation was blocked by a device security restriction, so the successful runtime evidence remains the Automotive emulator. This distinction has been retained in the report rather than presenting the phone attempt as a successful run.

USER INPUT
MEDIA PLAYER UI
MEDIA SERVICE
AUDIO FRAMEWORK
AUDIO HAL — conceptual
LOGCAT / VISIBLE STATE

Figure 17.1. Final conceptual command path represented by the project.

Final project status	Status
Android Automotive media-player UI	Implemented and demonstrated
Playback controls	Implemented
Conceptual Media Service / Audio HAL flow	Implemented through application logs
Automotive emulator testing	Demonstrated
Physical Xiaomi deployment	Attempted; installation restricted
GitHub repository	github.com/SAGNIK890/AutomotiveMediaPlayer

FINAL ENGINEERING STATEMENT

The project meets the intended educational objective of demonstrating an Android Automotive IVI media-player workflow with visible interaction, conceptual command propagation and runtime-oriented observation, while clearly identifying the boundaries of the prototype.